

Midterm — Sample Exam Solutions

Software Engineering — UATX — Spring 2026

Solutions and grading rubrics for the five sample questions. Try the questions cold first; the rubrics show what counts as a strong answer versus a partial one. Most questions have several valid approaches — the model answers below are one good version, not the only one.

QUESTION 1 — SOLUTION

JSON file, read-modify-write

A: 2 . After two `add()` calls, the file holds `a`, `b` .

B: 2 . `x.append(...)` mutates the local list `x` , but `load()` re-reads the file each time and gets back the original two items. *Why:* `load()` always re-reads from disk and returns a fresh list; in-memory mutations are not persisted unless explicitly written.

C: 3 . `save(x)` writes the list (now of length 3) back to disk.

D: False . `y[0]["done"] = True` mutates a local list (a fresh `load()` result). Then `add("d")` internally does its own `load()` — getting a copy without `y` 's mutation — appends `d` , and writes that back to disk. `y` 's mutation never reaches the file. *Why:* the read-modify-write trap from class — the second writer overwrites the first writer's local change.

E: `['a', 'b', 'c', 'd']` .

D is the trap. The intuitive (wrong) answer is to think `y` is some kind of live reference into the file, so `y[0]["done"] = True` persists. It doesn't — `y` is just a plain Python list returned by `load()` , completely independent of the file once read. The next writer (`add("d")`) does its own `load()` , doesn't see your local change, and overwrites it on save.

Rubric (10 pts): 1 each for A, B, C, E (4 pts). 2 for D's value, 2 for D's explanation. 2 for B's explanation.

List-posts endpoint defects

At least five real defects. Full credit for finding four; partial for fewer.

1. **SQL injection via `q`**. The `q` parameter is concatenated into the SQL with no escaping.

- (a) `?q=' OR '1'='1`
- (b) Final SQL becomes `... WHERE body LIKE '%' OR '1'='1%` — returns every row regardless of search term.
- (c) Use a bound parameter: `... WHERE body LIKE :pattern` with `{"pattern": f"%{q}%"}`.

2. **LIKE wildcard injection**. Even after fixing the SQL injection, the user controls the LIKE wildcard characters `%` and `_` by including them in `q`.

- (a) `?q=%` (a single percent sign, nothing else)
- (b) The code wraps `q` as `'%' + q + '%'`, so the pattern becomes `'%%%'`. In SQL, `%` is the LIKE wildcard meaning "any sequence of characters," so this pattern matches every row in the table — the search filter is gone. Parameter binding (the fix for the previous defect) doesn't help here, because the wildcard is the literal value the user submitted.
- (c) Before wrapping, escape `%` and `_` in `q` (e.g. replace `%` with `\%`) and add `ESCAPE '\'` to the LIKE clause.

3. **No upper bound on `limit`**.

- (a) `?limit=100000000`
- (b) Server tries to read 10M rows in one shot — memory blows up; trivial DoS.
- (c) Cap `limit` (e.g. `min(limit, 200)`); reject obviously bad values.

4. **`page=0` or negative pages produce a negative offset**.

- (a) `?page=0` or `?page=-2`
- (b) Computed offset is negative; SQLite silently treats negative OFFSET as 0, so page 0 and page 1 return the same data — pagination is wrong.
- (c) Validate `page ≥ 1` via `Query(..., ge=1)`.

5. **Negative `limit`**.

- (a) `?limit=-1`
- (b) SQLite treats negative LIMIT as "no limit" — returns the entire table.
- (c) Validate `limit ≥ 1` as well as the upper bound.

Other defects worth flagging (and credited if you found them): no `response_model`, so the response shape is whatever the DB happens to return; raw `body` exposed without consideration of soft-deleted or admin-flagged posts; no auth check on what should arguably be a private feed.

Rubric (10 pts): 2 pts per distinct defect with all three parts (a/b/c), capped at 8 points. 2 bonus pts for finding either the LIKE wildcard or the negative-limit issue (the two non-obvious ones). Total capped at ten.

QUESTION 3 — SOLUTION

Photo album sharing API

Many valid answers; the rubric is on completeness and judgment.

Endpoints (a reasonable answer):

```
POST    /api/albums                201 + Album, 400 invalid body
GET     /api/albums                200 + list[Album]
GET     /api/albums/{id}          200 + Album, 404
DELETE  /api/albums/{id}          204, 404
POST    /api/albums/{id}/photos   201 + Photo, 404 album not found, 400 bad
upload
DELETE  /api/albums/{id}/photos/{pid} 204, 404
POST    /api/albums/{id}/share    201 + ShareLink, 404, 409 already shared
DELETE  /api/albums/{id}/share    204, 404
GET     /shared/{token}           200 + AlbumView (no auth required), 404 /
410 if revoked
```

Strong answers nest photos under albums (the URL structure shows the relationship), use `POST` for share generation (since each call may produce a new token), and use `DELETE` for revoke.

Idempotence: The cleanest example is `DELETE /api/albums/{id}/share` (revoke) — revoking twice is a no-op (or returns 404), and a retry under a flaky network can't accidentally do more than one revoke. By contrast, `POST /api/albums` is NOT idempotent — retries create duplicate albums. Either example is a fine answer as long as you state the reason.

Design call (good answers):

- "Share link uses a UUID, not the album ID, so the URL is unguessable."
- "Revoking a share link generates a new token next time you share — old URLs stay 410-Gone forever."
- " `GET /shared/{token}` returns the album in a 'view-only' shape (no edit affordances, no other-user data) instead of the full `Album`."
- "Photos belong to one album — we don't support a photo in multiple albums."

Rubric (10 pts): 5 endpoints (need at least 6 of the listed; status codes for success and failure). 3 idempotence question. 2 design call (must be non-obvious).

QUESTION 4 — SOLUTION

`flag_expired` : loop vs comprehension

Version B is correct, and there is a real bug in A.

The bug: Version A mutates the input. `items[i]["expired"] = True` sets the key on the ORIGINAL dict in the input list, not a copy. The caller's `items` list now has `expired: True` on the matched dicts, even though the function only "promised" to return a filtered list. This bites a caller who calls `flag_expired` twice or who relies on the original list being unchanged.

Version B uses dict unpacking (`{**item, "expired": True}`) to build NEW dicts, leaving the original untouched.

Other reasons B wins (after the bug is named): more readable, no manual index, no accumulator list, harder to write the wrong loop bound, and the "filter-and-transform" comprehension is the idiomatic Python shape for deriving a new collection from an existing one without altering the source.

The strongest answer flags the mutation explicitly. Picking B for "it's more readable" and missing the mutation only gets you partial credit.

Rubric (10 pts): 2 picking B, 4 for calling out the mutation bug (2 noticing, 2 explaining what breaks), 2 for a second specific reason, 2 for quality of writing. Ceiling is 5/10 if you don't name the mutation.

Order placement, idempotence, retries

PATCH on a specific shipping address is idempotent: applying the same shipping update twice leaves the order in the same final state. **POST is not:** each retry creates a new order, charges payment again, and decrements inventory again. A flaky network turns one order into two or three, with the customer charged each time.

The fix for POST: have the client generate a unique id per send-attempt (a UUID) and let the server use it to deduplicate retries. Two clean ways to do this:

1. Switch the URL shape to `PUT /api/orders/{client_uuid}`, where the client supplies the id. The first `PUT` creates the order at that id; a retry hits the same row and is a no-op (or returns the same order). The endpoint is now naturally idempotent because the resource id is fixed by the client.
2. Keep `POST /api/orders` but require the client to send the UUID in the request body (e.g. `{"client_request_id": "...", "items": [...]}`). The server stores the id alongside the resulting order; on a second request with the same id, it returns the existing order without re-charging or re-decrementing. (Stripe's API uses this pattern via their idempotency keys; their dedup window is 24 hours.)

Either move solves the problem. The principle is the same: the client picks something stable that survives retries, and the server uses it to recognize "this is the same request twice," not "this is two different requests."

One trap to avoid: don't conflate idempotence with safety. `GET` is both safe (no side effects) and idempotent. `PUT` and `DELETE` are idempotent but not safe. `POST` is neither — which is why naked `POST` + retry is the dangerous combination here.

Rubric (10 pts): 3 correctly identifying which is idempotent (must reason, not just guess), 2 explaining what specifically breaks on POST retry (double-charge, double-decrement), 4 the design fix (must name a client-supplied id used for dedup — either as the URL slug or in the body; no credit for vague "retry once"), 1 the operational detail (e.g. dedup window, what the server stores).